UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación · Carrera de Software

Llamada a procedimiento remoto (RPC) construida desde cero en Java

Guía de práctica de laboratorio

Aplicaciones Distribuidas (ISR-701) · SPA 2026–2027

Campo	Detalle
Unidad y tema	Unidad 1 — Generalidades de los sistemas distribuidos · Modelos y comunicación; *middleware*
Semana	3 (9 al 15 de noviembre de 2026)
Duración	3 horas de práctica en laboratorio, más los retos de afianzamiento como trabajo autónomo
Modalidad	Individual; el reto 2 se hace con un compañero porque requiere dos computadoras
Relación con otras actividades	Prepara GA-SUM-01, Proyecto Experimental de la Unidad 1 (PE-U1): *sockets* y gRPC (semana 4)
Entorno	Windows 10 u 11 (64 bits), IntelliJ IDEA 2025.3.1 y JDK 21 (LTS). Sin Maven, Gradle ni bibliotecas externas. Detalle en la sección 3
Docente	Gleiston C. Guerrero-Ulloa, PhD (`gguerrero@uteq.edu.ec`)

1. Propósito

En un programa normal, llamar a una función es inmediato: los parámetros se pasan en memoria y el resultado vuelve en la misma máquina. La llamada a procedimiento remoto (RPC, por sus siglas en inglés, *Remote Procedure Call*) permite escribir esa misma llamada cuando la función se ejecuta en otra computadora. La idea fue formalizada por Birrell y Nelson, que la organizaron alrededor de unos componentes llamados *stubs* (piezas que se hacen pasar por la función real) y de un módulo de comunicación que transporta los mensajes [1]. No es una idea del pasado: un estudio de casi dos años sobre la infraestructura de Google analizó más de diez mil métodos RPC distintos y encontró que el procesamiento de estas llamadas consume alrededor del 7 % de todos los ciclos de CPU de sus centros de datos [2].

En esta práctica no se usa ningún *framework*: cada estudiante construye a mano todas las piezas de RPC con *sockets* y la serialización estándar de Java. Así verá qué hacen por dentro herramientas como gRPC o Java RMI, que la semana siguiente se usarán en el PE-U1.

Al terminar la práctica el estudiante será capaz de

1. Identificar los componentes de la arquitectura RPC (contrato, *stub* del cliente, *stub* del servidor y runtime de comunicación) y ubicar cada uno en su propio código.
2. Implementar en Java un servicio remoto con *sockets* TCP y serialización de objetos.

3. Ejecutar cliente y servidor en dos computadoras distintas.
4. Explicar por qué una llamada remota no es igual a una llamada local, en especial ante fallos de red.
5. Relacionar cada pieza construida a mano con su equivalente en gRPC.

2. Fundamentos mínimos

2.1. Componentes de la arquitectura

La figura 1 muestra las piezas que se construirán. Lo que el programador escribe (en azul) es lo mismo que escribiría en un programa local. Lo que hace posible la distribución (en verde azulado y gris) queda escondido debajo [1], [3].

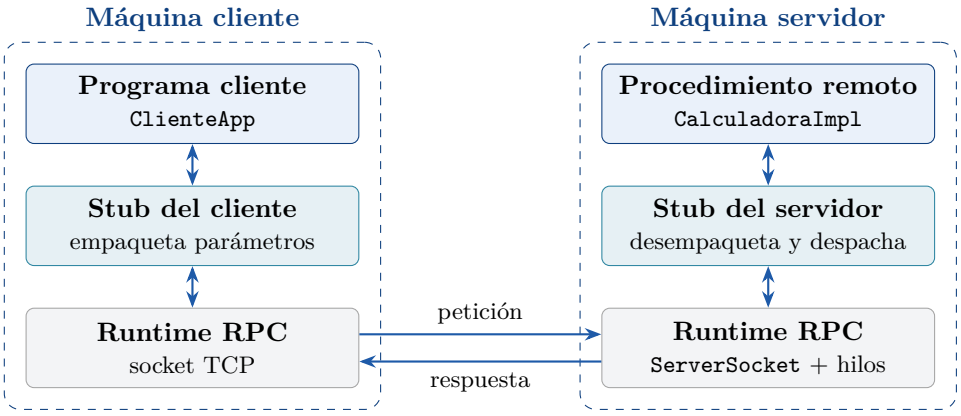


Figura 1: Arquitectura RPC que se implementa en esta práctica

Componente	Qué hace	Archivo en la práctica
Contrato (o IDL, lenguaje de definición de interfaces)	Declara qué procedimientos existen, con qué parámetros y qué devuelven. Cliente y servidor lo comparten.	`Calculadora.java`
Mensajes	Llevan por la red el nombre del procedimiento, los parámetros y el resultado.	`Peticion.java` `Respuesta.java`
Stub del cliente (o *proxy*)	Se hace pasar por el procedimiento real: recibe la llamada local, la empaqueta, la envía y espera la respuesta.	`ClienteStub.java`
Stub del servidor (o *skeleton*)	Recibe la petición, averigua qué procedimiento se pidió, extrae los parámetros y llama a la implementación.	`ServidorStub.java`
Runtime RPC	Transporta los bytes: abre conexiones, envía, recibe y atiende a varios clientes.	`ServidorRPC.java` y el socket dentro de `ClienteStub.java`
Implementación	El código que realmente hace el trabajo. Solo existe en el servidor.	`CalculadoraImpl.java`

2.2. Recorrido de una llamada

Cuando el cliente escribe `calc.sumar(7, 5)` ocurren estos pasos, en este orden [3]:

1. El programa cliente llama a `sumar` en el *stub* del cliente como si fuera un método local.
2. El *stub* construye un mensaje con el nombre `"sumar"` y los parámetros `7` y `5`.
3. El mensaje se convierte en una secuencia de bytes. A esta conversión se le llama empaquetado de parámetros (o *marshalling*).
4. El runtime envía los bytes por la red y el cliente queda bloqueado esperando.
5. En el servidor, el runtime recibe los bytes y reconstruye el mensaje. A esta reconstrucción se le llama desempaquetado (o *unmarshalling*).
6. El *stub* del servidor lee el nombre `"sumar"`, convierte los parámetros a `int` y llama a la implementación real.
7. La implementación devuelve `12`.
8. El *stub* del servidor empaqueta la respuesta y el runtime la envía.
9. El *stub* del cliente desempaqueta la respuesta.
10. El *stub* devuelve `12` al programa cliente, que continúa como si nada hubiera ocurrido.

Cómo se aplica en la práctica: en el código, los pasos 2 a 4 están dentro del método `invocar` de `ClienteStub`, los pasos 5 y 8 en el bucle de `ServidorRPC.atender`, y el paso 6 en `ServidorStub.despachar`. Al terminar, el estudiante debería poder señalar cada paso con el dedo sobre su propio código.

2.3. El límite de la transparencia

El objetivo de RPC es que la llamada remota se vea igual que la local; a esto se le llama transparencia de acceso [3]. Pero Waldo y colegas advirtieron que tratar ambas llamadas como iguales es un error de diseño, porque una llamada remota tiene una latencia mucho mayor y puede fallar a medias: el cliente puede quedarse sin saber si el servidor ejecutó o no la operación [4]. Esta práctica deja ver ese problema en el reto 3.

Cómo se aplica en la práctica: observe que `ClienteStub.invocar` puede lanzar `ErrorRemoto`, algo que un método local como `a + b` nunca haría. El programa cliente debe estar preparado para ese caso aunque la llamada «parezca» local.

3. Entorno de trabajo y versiones

3.1. Versiones utilizadas

La práctica se preparó y se probó con las versiones de la tabla 1. Use exactamente estas versiones para que lo que vea en pantalla coincida con la guía.

Tabla 1: Versiones de las herramientas de la práctica

Elemento	Versión	Observación
Sistema operativo	Windows 10 u 11, de 64 bits	Las órdenes de consola de la guía son de Windows (PowerShell).
Entorno de desarrollo (IDE)	IntelliJ IDEA 2025.3.1	Desde la versión 2025.3 existe un solo producto, que reemplaza a las ediciones Community y Ultimate; las funciones básicas para Java son gratuitas y no requieren activación [5], [6].
Kit de desarrollo de Java (JDK)	JDK 21 (versión de soporte extendido, LTS), cualquier actualización 21.0.x	Java SE 21 es una versión LTS publicada en septiembre de 2023 [7]. Se recomienda la distribución Eclipse Temurin 21, que IntelliJ IDEA descarga por usted (sección 3.3). El código de la guía se probó con OpenJDK 21.0.12.
Nivel de lenguaje	21	Se usan registros (`record`, desde Java 16), expresiones `switch` (desde Java 14) y filtros de serialización (desde Java 9). Nada exige una versión posterior a la 21.
Sistema de construcción	Ninguno (opción *IntelliJ* del asistente)	No se usa Maven ni Gradle, ni bibliotecas externas: todo está en el JDK.
Red	TCP, puerto 5000	Cliente y servidor en la misma computadora (`localhost`) o en la misma red local (reto 2).

Dos «Java» distintos

IntelliJ IDEA trae su propio entorno de ejecución (JetBrains Runtime) para funcionar él mismo, pero ese entorno no es el JDK del proyecto. Para compilar y ejecutar el código de la práctica se necesita un JDK 21 configurado en el proyecto, como se indica a continuación [8].

3.2. Instalar IntelliJ IDEA 2025.3.1

1. Abra la página de versiones anteriores de IntelliJ IDEA (`https://www.jetbrains.com/idea/download/other/`) [9], elija la versión **2025.3.1** y descargue el instalador para Windows (archivo `.exe`).
2. Ejecute el instalador y acepte la carpeta propuesta. En la pantalla de opciones puede marcar *Create Desktop Shortcut*; las demás opciones no son necesarias para la práctica.
3. Abra IntelliJ IDEA. Si aparece la pantalla de bienvenida (*Welcome*), continúe con la sección siguiente. Si ya tenía instalada otra versión, compruebe la versión en **Help | About**: debe decir `IntelliJ IDEA 2025.3.1`.

3.3. Crear el proyecto con el JDK 21

1. En la pantalla de bienvenida haga clic en **New Project**. Si ya hay un proyecto abierto, use **File | New | Project** [10].
2. En la lista de la izquierda seleccione **Java** y complete:
   - **Name**: `rpc-java`
   - **Location**: una carpeta propia, por ejemplo `C:\Practicas`
   - **Create Git repository**: desmarcado (puede activarlo si desea versionar su trabajo)
   - **Build system**: **IntelliJ**
   - **JDK**: elija un JDK 21 si aparece en la lista. Si no aparece, elija **Download JDK...**; en el cuadro siguiente seleccione la versión **21** y el proveedor **Eclipse Temurin**, deje la ruta propuesta y confirme. IntelliJ descarga e instala el JDK [8].
   - **Add sample code**: desmarcado
3. Haga clic en **Create**. Se crea el proyecto con una carpeta `src` vacía.
4. Compruebe la configuración en **File | Project Structure** (`Ctrl+Alt+Mayús+S`), apartado **Project**: **SDK** debe mostrar el JDK 21 y **Language level** debe ser **21** (o *SDK default*, que con el JDK 21 equivale a 21). Cierre con **OK**.

3.4. Crear cada archivo de la práctica

Las clases se crean en la carpeta `src`, sin paquete (`package`), para que el proyecto sea lo más simple posible:

1. En la ventana del proyecto (*Project*, `Alt+1`), haga clic derecho sobre `src` y elija **New | Java Class**.
2. Escriba el nombre exacto de la clase (por ejemplo, `Calculadora`) y, en la lista de tipos, elija **Interface** para `Calculadora`, **Record** para `Peticion` y `Respuesta`, y **Class** para las demás. Pulse `Enter`.
3. Reemplace el contenido generado por el código de cada paso. El nombre del archivo debe coincidir exactamente con el de su clase pública.

Al terminar el paso 7, la carpeta `src` debe contener ocho archivos: `Calculadora`, `Peticion`, `Respuesta`, `CalculadoraImpl`, `ServidorStub`, `ServidorRPC`, `ClienteStub` y `ClienteApp`. La excepción `ErrorRemoto` no tiene archivo propio: es una clase anidada dentro de `ClienteStub`.

4. Desarrollo paso a paso

Escriba cada archivo usted mismo, leyendo los comentarios. Copiar y pegar todo de una vez quita el sentido de la práctica: lo importante es entender qué papel cumple cada línea en la figura 1.

4.1. Paso 1. Definir el contrato

La interfaz es el acuerdo entre cliente y servidor. En gRPC este papel lo cumple un archivo `.proto` [11]; aquí usamos una interfaz de Java.

src/Calculadora.java

```
// PASO 1. Contrato del servicio (equivale a la IDL del RPC clasico).
// Cliente y servidor comparten esta interfaz; ninguno conoce el codigo del otro.
public interface Calculadora {
```

```
    int sumar(int a, int b);
    double dividir(double a, double b);
    String saludar(String nombre);
}
```

4.2. Paso 2. Definir los mensajes

La petición lleva el nombre del método y un arreglo con los parámetros; la respuesta lleva el valor o el texto del error. Ambos implementan `Serializable` para que `ObjectOutputStream` pueda convertirlos en bytes.

src/Peticion.java

```
import java.io.Serializable;

// PASO 2. Mensaje de peticion: nombre del procedimiento y sus parametros.
// Serializable permite convertirlo en bytes (marshalling) para enviarlo por la red.
public record Peticion(String metodo, Object[] args) implements Serializable {
}
```

src/Respuesta.java

```
import java.io.Serializable;

// PASO 2. Mensaje de respuesta: valor devuelto o descripcion del error remoto.
public record Respuesta(Object valor, String error) implements Serializable {
    public static Respuesta ok(Object valor) { return new Respuesta(valor, null); }
    public static Respuesta fallo(String error) { return new Respuesta(null, error); }
    public boolean esError() { return error != null; }
}
```

Qué observar: el error viaja como texto dentro de la respuesta. Una excepción no puede «saltar» de una máquina a otra; hay que transportarla como dato y volver a lanzarla del otro lado.

4.3. Paso 3. Implementar el procedimiento real

src/CalculadoraImpl.java

```
// PASO 3. Implementacion real del procedimiento remoto (vive solo en el servidor).
public class CalculadoraImpl implements Calculadora {
    @Override
    public int sumar(int a, int b) {
        return a + b;
    }

    @Override
    public double dividir(double a, double b) {
        if (b == 0) {
            throw new ArithmeticException("division entre cero");
        }
        return a / b;
    }

    @Override
    public String saludar(String nombre) {
        return "Hola, " + nombre + ", te saluda el servidor RPC";
    }
}
```

La excepción de `dividir` es intencional: servirá para comprobar que los errores del servidor llegan al cliente.

4.4. Paso 4. Escribir el stub del servidor

El *skeleton* traduce un mensaje genérico en una llamada concreta. Si el nombre del método no existe, responde con un error en lugar de caerse.

src/ServidorStub.java

```java
// PASO 4. Stub del servidor (skeleton): recibe la peticion ya deserializada,
// identifica el procedimiento, extrae los parametros y llama a la implementacion.
public class ServidorStub {
    private final Calculadora impl;

    public ServidorStub(Calculadora impl) {
        this.impl = impl;
    }

    public Respuesta despachar(Peticion p) {
        try {
            Object[] a = p.args();
            Object resultado = switch (p.metodo()) {
                case "sumar" -> impl.sumar((Integer) a[0], (Integer) a[1]);
                case "dividir" -> impl.dividir((Double) a[0], (Double) a[1]);
                case "saludar" -> impl.saludar((String) a[0]);
                default -> throw new IllegalArgumentException("procedimiento
    desconocido: " + p.metodo());
            };
            return Respuesta.ok(resultado);
        } catch (Exception e) {
            return Respuesta.fallo(e.getClass().getSimpleName() + ": " + e.getMessage()
    );
        }
    }
}
```

Qué observar: el `switch` hay que escribirlo a mano para cada método. Esta es la parte repetitiva que gRPC genera automáticamente a partir del contrato.

4.5. Paso 5. Escribir el runtime del servidor

src/ServidorRPC.java

```java
import java.io.EOFException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.ServerSocket;
import java.net.Socket;

// PASO 5. Runtime del servidor: escucha en un puerto TCP, atiende cada cliente
// en su propio hilo y entrega cada peticion al stub del servidor.
public class ServidorRPC {
    public static void main(String[] args) throws Exception {
        int puerto = args.length > 0 ? Integer.parseInt(args[0]) : 5000;
        ServidorStub stub = new ServidorStub(new CalculadoraImpl());
```

```
        try (ServerSocket servidor = new ServerSocket(puerto)) {
            System.out.println("[Servidor] Escuchando en el puerto " + puerto);
            while (true) {
                Socket cliente = servidor.accept();
                new Thread(() -> atender(cliente, stub)).start();
            }
        }
    }

    private static void atender(Socket socket, ServidorStub stub) {
        String quien = socket.getRemoteSocketAddress().toString();
        System.out.println("[Servidor] Cliente conectado: " + quien);
        try (socket;
             ObjectOutputStream salida = new ObjectOutputStream(socket.getOutputStream
    ());
             ObjectInputStream entrada = new ObjectInputStream(socket.getInputStream())
    ) {
            while (true) {
                Peticion p = (Peticion) entrada.readObject();      // unmarshalling
                System.out.println("[Servidor] Peticion " + p.metodo() + " de " +
    quien);
                Respuesta r = stub.despachar(p);
                salida.writeObject(r);                              // marshalling
                salida.flush();
            }
        } catch (EOFException fin) {
            System.out.println("[Servidor] Cliente desconectado: " + quien);
        } catch (Exception e) {
            System.out.println("[Servidor] Error con " + quien + ": " + e);
        }
    }
}
```

Qué observar: cada cliente se atiende en un hilo propio, así que varios clientes pueden llamar al mismo tiempo. Esto tendrá consecuencias en el reto 1, cuando el servidor guarde un dato compartido.

4.6. Paso 6. Escribir el stub del cliente

src/ClienteStub.java

```
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.Socket;

// PASO 6. Stub del cliente (proxy): implementa la MISMA interfaz que el servidor,
// de modo que el programa cliente cree que llama a un metodo local.
// Por dentro empaqueta la llamada, la envia, espera y desempaqueta la respuesta.
public class ClienteStub implements Calculadora, AutoCloseable {

    public static class ErrorRemoto extends RuntimeException {
        private static final long serialVersionUID = 1L;

        public ErrorRemoto(String mensaje) { super(mensaje); }
    }

    private final Socket socket;
```

```
    private final ObjectOutputStream salida;
    private final ObjectInputStream entrada;

    public ClienteStub(String host, int puerto) throws IOException {
        socket = new Socket(host, puerto);
        salida = new ObjectOutputStream(socket.getOutputStream());
        salida.flush();
        entrada = new ObjectInputStream(socket.getInputStream());
    }

    private synchronized Object invocar(String metodo, Object... args) {
        try {
            salida.writeObject(new Peticion(metodo, args));   // marshalling
            salida.reset();
            salida.flush();
            Respuesta r = (Respuesta) entrada.readObject();   // espera bloqueante
            if (r.esError()) {
                throw new ErrorRemoto(r.error());
            }
            return r.valor();
        } catch (IOException | ClassNotFoundException e) {
            throw new ErrorRemoto("fallo de comunicacion: " + e.getMessage());
        }
    }

    @Override
    public int sumar(int a, int b) {
        return (Integer) invocar("sumar", a, b);
    }

    @Override
    public double dividir(double a, double b) {
        return (Double) invocar("dividir", a, b);
    }

    @Override
    public String saludar(String nombre) {
        return (String) invocar("saludar", nombre);
    }

    @Override
    public void close() throws IOException {
        socket.close();
    }
}
```

Qué observar: el cliente crea primero el `ObjectOutputStream` y lo vacía con `flush()` antes de crear el `ObjectInputStream`. Si ambos extremos crean primero el flujo de entrada, cada uno espera la cabecera del otro y el programa se queda congelado. La llamada a `reset()` evita que Java reutilice referencias a objetos ya enviados.

4.7. Paso 7. Escribir el programa cliente

src/ClienteApp.java

```
// PASO 7. Programa cliente: solo conoce la interfaz Calculadora.
// Las llamadas se escriben igual que si fueran locales (transparencia de acceso).
public class ClienteApp {
```

```
    public static void main(String[] args) throws Exception {
        String host = args.length > 0 ? args[0] : "localhost";
        int puerto = args.length > 1 ? Integer.parseInt(args[1]) : 5000;

        try (ClienteStub stub = new ClienteStub(host, puerto)) {
            Calculadora calc = stub;

            System.out.println("sumar(7, 5)      = " + calc.sumar(7, 5));
            System.out.println("dividir(10, 4)   = " + calc.dividir(10, 4));
            System.out.println("saludar(\"UTEQ\") = " + calc.saludar("UTEQ"));

            try {
                calc.dividir(1, 0);
            } catch (ClienteStub.ErrorRemoto e) {
                System.out.println("dividir(1, 0)    -> error remoto: " + e.getMessage
    ());
            }
        }
    }
}
```

4.8. Paso 8. Compilar y ejecutar en IntelliJ IDEA

1. **Compilar.** Elija **Build | Build Project** (`Ctrl+F9`). La barra de estado debe indicar que la compilación terminó sin errores. Los archivos `.class` quedan en `out\production\rpc-java`.
2. **Iniciar el servidor.** Abra `ServidorRPC`, haga clic en la flecha verde que aparece en el margen junto a `main` y elija **Run 'ServidorRPC.main()'** [10]. En la ventana *Run* (`Alt+4`) debe aparecer `[Servidor] Escuchando en el puerto 5000`. El servidor queda en ejecución.
3. **Permitir el acceso en el firewall.** La primera vez que el servidor abre el puerto, Windows puede mostrar una alerta de seguridad que pregunta si se permite el acceso a la red al programa Java (*OpenJDK Platform binary*). Permítalo solo en redes privadas.
4. **Iniciar el cliente.** Abra `ClienteApp` y ejecútelo de la misma forma. En la ventana *Run* aparece una pestaña nueva para el cliente; la del servidor sigue abierta y registra cada petición.
5. **Pasar argumentos (opcional en este paso, necesario en el reto 2).** Elija **Run | Edit Configurations**, seleccione `ClienteApp` y escriba en **Program arguments**: `localhost 5000`. Para el servidor, en su configuración escriba `5000`. Sin argumentos, ambos programas usan esos mismos valores por defecto [12].
6. **Detener el servidor.** Seleccione su pestaña en la ventana *Run* y pulse el cuadrado rojo *Stop* (`Ctrl+F2`).

Dos clientes a la vez

Por defecto, IntelliJ IDEA ejecuta una sola instancia de cada configuración: si vuelve a ejecutar `ClienteApp` mientras sigue activa, primero detiene la anterior. Para lanzar dos clientes simultáneos abra **Run | Edit Configurations**, seleccione `ClienteApp`, pulse **Modify options** y active **Allow multiple instances** [12]. El servidor no lo necesita: debe haber uno solo escuchando en el puerto.

Alternativa desde la terminal. La terminal integrada de IntelliJ IDEA (**View | Tool Windows | Terminal**, `Alt+F12`) se abre en la carpeta del proyecto. Con el JDK 21 en el `PATH`, las mismas clases se compilan y ejecutan así, en dos terminales:

```
javac --release 21 -d out\production\rpc-java src\*.java
```

```
java -cp out\production\rpc-java ServidorRPC 5000
java -cp out\production\rpc-java ClienteApp localhost 5000
```

Esta forma es útil en la computadora del compañero del reto 2 si allí solo se quiere ejecutar el cliente. Compruebe antes la versión con `java -version` y `javac -version`: ambas deben empezar por `21`.

La salida esperada en la pestaña del cliente es:

```
sumar(7, 5)      = 12
dividir(10, 4)   = 2.5
saludar("UTEQ") = Hola, UTEQ, te saluda el servidor RPC
dividir(1, 0)    -> error remoto: ArithmeticException: division entre cero
```

En la pestaña del servidor debe aparecer una línea `Peticion` por cada llamada y, al final, `Cliente desconectado`.

Comprobación antes de seguir

- ☐ El cliente muestra los cuatro resultados, incluido el error remoto de la división entre cero.
- ☐ El servidor registra cuatro peticiones y la desconexión del cliente.
- ☐ Puedo señalar en mi código dónde ocurre cada uno de los diez pasos del recorrido de una llamada.
- ☐ Si detengo el servidor y ejecuto el cliente, obtengo `java.net.ConnectException` (conexión rechazada) y entiendo por qué.

5. Retos de afianzamiento

Los retos amplían el sistema base y se resuelven en el mismo proyecto. Cada uno termina con una pregunta para pensar; la respuesta no está en el código, sino en lo que se observa al ejecutarlo.

Reto 1. Ampliar el contrato y guardar estado en el servidor

Agregue dos operaciones al contrato: `int multiplicar(int a, int b)` y `long llamadasAtendidas()`, que devuelve cuántas llamadas ha atendido el servidor desde que arrancó. Recorra los cuatro archivos que deben cambiar: contrato, implementación, *stub* del servidor y *stub* del cliente.

Pista: el contador lo comparten todos los hilos del servidor. Escríbalo primero como `private long contador` con `contador++` y ejecute ocho clientes en paralelo con 500 llamadas cada uno (cree una clase `ClienteConcurrente` que lance ocho hilos, cada uno con su propio `ClienteStub`). Compare el resultado con 4 000. Luego cámbielo por `java.util.concurrent.atomic.AtomicLong` y repita.

Pregunta para pensar: ¿por qué `contador++` puede perder llamadas si es una sola línea de código?

Reto 2. Distribución real en dos computadoras

Ejecute el servidor en su computadora y el cliente en la de un compañero conectado a la misma red.

1. En la computadora del servidor, averigüe su dirección IPv4 con `ipconfig`.
2. Permita conexiones entrantes al puerto 5000 en el Firewall de Windows. En una ventana de

PowerShell abierta con *Ejecutar como administrador*:

```
netsh advfirewall firewall add rule name="RPC-5000" dir=in action=allow protocol
    =TCP localport=5000
```

3. En la otra computadora ejecute el cliente con esa dirección como primer argumento: en IntelliJ IDEA, **Program arguments** `192.168.x.y 5000`; o en la terminal, `java -cp out\production\rpc-java ClienteApp 192.168.x.y 5000`.
4. Al terminar, elimine la regla:

```
netsh advfirewall firewall delete rule name="RPC-5000"
```

Haga que `saludar` devuelva también el nombre del equipo servidor, obtenido con `InetAddress.getLocalHost().getHostName()` (clase del paquete `java.net`), para demostrar que la respuesta vino de la otra máquina.

Pregunta para pensar: el código del cliente no cambió en absoluto. ¿Qué tipo de transparencia demuestra eso, y qué información sí tuvo que conocer el usuario para que funcionara?

Reto 3. Tiempo límite y fallos a medias

Agregue al contrato `String lenta(int milisegundos)`: la implementación imprime un mensaje al empezar, espera con `Thread.sleep(milisegundos)`, imprime otro mensaje al terminar y devuelve un texto. En el constructor de `ClienteStub` fije un tiempo máximo de espera de 2 segundos con `socket.setSoTimeout(2000)`. Según la documentación de Java, si la respuesta no llega en ese tiempo, la lectura lanza `SocketTimeoutException`, aunque el socket sigue siendo válido [13]. Capture esa excepción en `invocar` y conviértala en un `ErrorRemoto` con un mensaje claro. Llame a `lenta(4000)` y observe las dos ventanas.

Lo que va a ver: el cliente abandona a los 2 segundos, pero el servidor imprime «terminó» a los 4 segundos. La operación sí se ejecutó, aunque el cliente cree que falló.

Pregunta para pensar: si en lugar de `lenta` la operación fuera `depositar(cuenta, 100)`, ¿sería seguro que el cliente la reintente automáticamente? Use la tabla 2 para responder y explique por qué reintentar `sumar` sí es seguro (la operación es idempotente: repetirla no cambia el resultado).

Tabla 2: Semánticas de una llamada remota ante fallos [3]

Semántica	Qué garantiza	Cómo se logra en la práctica
Al menos una vez (*at-least-once*)	La operación se ejecutó una o más veces.	El cliente reintenta hasta recibir respuesta. Solo es segura con operaciones idempotentes.
Como máximo una vez (*at-most-once*)	La operación se ejecutó una vez o ninguna, nunca dos.	El cliente no reintenta, o el servidor guarda un identificador de cada petición y descarta las repetidas.

Semántica	Qué garantiza	Cómo se logra en la práctica
Exactamente una vez (*exactly-once*)	Lo que quisiéramos: una sola ejecución siempre.	En general no puede garantizarse si el servidor puede caerse; se aproxima con identificadores de petición y almacenamiento persistente.

Reto 4. No aceptar cualquier objeto que llegue por la red

El servidor deserializa todo lo que recibe. La guía oficial de Java advierte que deserializar datos no confiables es peligroso, porque puede ejecutar código de clases arbitrarias, y recomienda usar filtros de serialización, que no vienen activados por defecto [14]. Agregue un filtro de lista blanca justo después de crear el `ObjectInputStream` del servidor:

```
ObjectInputFilter filtro = ObjectInputFilter.Config.createFilter(
        "Peticion;java.lang.*;maxdepth=5;!*");
entrada.setObjectInputFilter(filtro);

```

El patrón acepta la clase `Peticion` y las clases de `java.lang` (`String`, `Integer`, `Double`), limita la profundidad del objeto y rechaza todo lo demás (`!*`). Para probarlo, escriba un cliente que envíe un `java.util.HashMap` y capture en el servidor la excepción `InvalidClassException`.

Pregunta para pensar: ¿por qué una lista blanca (aceptar solo lo conocido) es más segura que una lista negra (rechazar lo que se sabe peligroso)?

Reto 5. De este RPC a gRPC

gRPC define el servicio en un archivo `.proto`, genera a partir de él el *stub* del cliente y el código del servidor, y permite fijar un plazo máximo a cada llamada [11]. Complete la tabla 3 con lo que observó en su código y escriba el archivo `calculadora.proto` equivalente a su interfaz `Calculadora` final (con los mensajes de entrada y salida de cada método). No hace falta compilarlo todavía: eso se hará en el PE-U1.

Pregunta para pensar: ¿qué partes de su código desaparecerían si usara gRPC, y cuáles tendría que seguir escribiendo usted?

Tabla 3: Correspondencia entre la práctica y gRPC

En esta práctica	En gRPC	Lo que observé (completar)
`Calculadora.java`	Servicio en el archivo `.proto`	
`Peticion`, `Respuesta`	Mensajes de *Protocol Buffers*	
Serialización de Java	Codificación binaria de *Protocol Buffers*	
`ClienteStub` escrito a mano	*Stub* generado por el compilador	
`switch` en `ServidorStub`	Clase base generada que se extiende	

En esta práctica	En gRPC	Lo que observé (completar)
`setSoTimeout(2000)`	Plazo de la llamada (*deadline*)	
`ErrorRemoto`	Códigos de estado (por ejemplo, `DEADLINE_EXCEEDED`)	

6. Preguntas de reflexión

1. Agregue un componente `long id` a `Peticion` solo en el servidor y ejecute un cliente con la versión anterior. No aparece ningún error: en los registros, la especificación de serialización de Java no exige que coincida el `serialVersionUID` y asigna el valor por defecto (aquí `0`) al componente que no viene en el flujo [15]. ¿Por qué un cambio que no produce error puede ser más peligroso que uno que sí lo produce?
2. El *stub* del cliente usa `synchronized` en `invocar`. ¿Qué podría pasar si dos hilos del cliente usaran el mismo *stub* sin esa palabra?
3. La serialización de Java solo la entienden programas escritos en Java. ¿Qué limitación impone eso a un sistema distribuido con clientes en Python o en un navegador, y cómo lo resuelve gRPC?
4. Según el estudio de Google, el costo de RPC en el tiempo total de una llamada es pequeño en promedio pero puede ser muy grande en los casos más lentos [2]. ¿Qué partes de su implementación (serialización, creación de hilos, red) cree que pesarían más si el servidor recibiera miles de llamadas por segundo?

7. Problemas frecuentes

Síntoma	Causa probable y solución
Aviso *Project JDK is not defined* en el editor	El proyecto no tiene JDK. Abra **File | Project Structure | Project** y elija el JDK 21 en **SDK**, o descárguelo con **Download JDK** (sección 3.3).
`javac` o `java` no se reconocen en la terminal	El JDK no está en el `PATH` de Windows. Dentro de IntelliJ IDEA no hace falta: use **Build** y **Run**. Para la terminal, agregue la carpeta `bin` del JDK 21 al `PATH` y reabra IntelliJ IDEA.
`error: release version 21 not supported`	La terminal está usando un JDK anterior al 21. Compruebe con `javac -version`.
`ConnectException` (conexión rechazada)	El servidor no está corriendo, escucha en otro puerto o la IP es incorrecta. Inicie primero el servidor y revise el puerto.
`BindException: Address already in use`	Ya hay un servidor usando el puerto 5000 (quizá otra pestaña de *Run* o una terminal olvidada). Deténgalo o use otro puerto como argumento.
El cliente se queda congelado al conectar	Ambos extremos crearon primero el `ObjectInputStream`. Cree y vacíe (`flush`) el `ObjectOutputStream` antes.

Síntoma	Causa probable y solución
`ClassCastException` en el servidor	El tipo de un parámetro enviado no coincide con la conversión del *skeleton*; por ejemplo, un *stub* del cliente que declara `dividir` con parámetros `int` en lugar de `double`. Revise que ambos usen la misma interfaz `Calculadora`.
Un parámetro llega como `0` o `null` sin explicación	Cliente y servidor se compilaron con versiones distintas de `Peticion`. Los registros no fallan en ese caso: el componente que falta recibe su valor por defecto [15]. Compile ambos lados con el mismo código.
Funciona en `localhost` pero no entre dos equipos	Firewall de Windows bloqueando el puerto, o equipos en redes distintas (por ejemplo, una red de invitados que aísla a los clientes).

8. Organización sugerida de la sesión

Tiempo	Actividad
0:00 – 0:25	Fundamentos: arquitectura, recorrido de una llamada y límite de la transparencia (sección 2).
0:25 – 0:40	Entorno: abrir IntelliJ IDEA 2025.3.1, crear el proyecto `rpc-java` con el JDK 21 y comprobar **Project Structure** (sección 3).
0:40 – 1:50	Pasos 1 a 7, guiados por las diapositivas, con una pausa de comprobación después del paso 4.
1:50 – 2:10	Paso 8: compilación, ejecución y lista de comprobación.
2:10 – 2:50	Retos 1 y 2 en el laboratorio (el reto 2 en parejas).
2:50 – 3:00	Cierre: los retos 3 a 5 y las preguntas de reflexión quedan como trabajo autónomo, como preparación del PE-U1.

Referencias

[1] A. D. Birrell y B. J. Nelson, «Implementing Remote Procedure Calls,» *ACM Transactions on Computer Systems*, vol. 2, n.º 1, págs. 39-59, 1984. DOI: 10.1145/2080.357392.

[2] K. Seemakhupt, B. E. Stephens, S. Khan et al., «A Cloud-Scale Characterization of Remote Procedure Calls,» en *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, New York, NY, USA: ACM, 2023, págs. 498-514. DOI: 10.1145/3600006.3613156.

[3] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.ª ed. Amsterdam, The Netherlands: Maarten van Steen, 2023. dirección: https://www.distributed-systems.net/.

[4] J. Waldo, G. Wyant, A. Wollrath y S. Kendall, «A Note on Distributed Computing,» en *Mobile Object Systems: Towards the Programmable Internet*, ép. Lecture Notes in Computer Science, J. Vitek y C. Tschudin, eds., vol. 1222, Berlin, Heidelberg: Springer, 1997, págs. 49-64. DOI: 10.1007/3-540-62852-5_6.

[5] JetBrains. «The Unified IntelliJ IDEA: More Free Features, a Better Experience, Smoother Flow,» The IntelliJ IDEA Blog. (2025), dirección: https://blog.jetbrains.com/idea/2025/12/intellij-idea-unified-release/ (visitado 12-11-2026).

[6] JetBrains. «IntelliJ IDEA 2025.3.1 Is Out!» The IntelliJ IDEA Blog. (2025), dirección: https://blog.jetbrains.com/idea/2025/12/intellij-idea-2025-3-1/ (visitado 12-11-2026).

[7] Oracle. «Oracle Java SE Support Roadmap.» (2026), dirección: https://www.oracle.com/java/technologies/java-se-support-roadmap.html (visitado 12-11-2026).

[8] JetBrains. «SDKs, IntelliJ IDEA Help.» (2026), dirección: https://www.jetbrains.com/help/idea/sdk.html (visitado 12-11-2026).

[9] JetBrains. «IntelliJ IDEA – Other Versions.» (2026), dirección: https://www.jetbrains.com/idea/download/other/ (visitado 12-11-2026).

[10] JetBrains. «Create your first Java application, IntelliJ IDEA Help.» (2026), dirección: https://www.jetbrains.com/help/idea/creating-and-running-your-first-java-application.html (visitado 12-11-2026).

[11] gRPC Authors. «Core concepts, architecture and lifecycle,» gRPC. (2026), dirección: https://grpc.io/docs/what-is-grpc/core-concepts/ (visitado 12-11-2026).

[12] JetBrains. «Run/debug configuration: Application, IntelliJ IDEA Help.» (2026), dirección: https://www.jetbrains.com/help/idea/run-debug-configuration-application.html (visitado 12-11-2026).

[13] Oracle. «Class Socket (Java SE 21 & JDK 21 API Specification).» (2023), dirección: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/Socket.html (visitado 12-11-2026).

[14] Oracle. «Addressing Deserialization Vulnerabilities, Java Core Libraries Developer Guide, Release 21.» (2023), dirección: https://docs.oracle.com/en/java/javase/21/core/addressing-serialization-vulnerabilities.html (visitado 12-11-2026).

[15] Oracle. «Java Object Serialization Specification: 1 – System Architecture, Section 1.13, Serialization of Records (Java SE 21).» (2023), dirección: https://docs.oracle.com/en/java/javase/21/docs/specs/serialization/serial-arch.html (visitado 12-11-2026).